

Zara E-Commerce Database Management System

DBMS – Payment Management system

1.Database design

1.1 Payment table

Attribute	Data Type	Constraint
Payment_ID	NUMBER	Primary Key
Order_ID	NUMBER	Foreign Key
Payment_Mode	VARCHAR2(20)	NOT NULL
Payment_Date	DATE	NOT NULL
Payment_Amount	NUMBER(10,2)	NOT NULL
Payment_Status	VARCHAR2(20)	NOT NULL

2.Table relationships

CUSTOMER

|

| places

▼

ORDERS

|

| has

▼

PAYMENT

ORDERS

|

| 1 : 1

▼

PAYMENT

3. Table creation

3.1 Create payment table

```
SQL> CREATE TABLE Payment
 2  (
 3      Payment_ID NUMBER PRIMARY KEY,
 4      Order_ID NUMBER,
 5      Payment_Mode VARCHAR2(20) NOT NULL,
 6      Payment_Date DATE NOT NULL,
 7      Payment_Amount NUMBER(10,2) NOT NULL,
 8      Payment_Status VARCHAR2(20) NOT NULL,
 9      CONSTRAINT fk_payment_order
10      FOREIGN KEY (Order_ID)
11      REFERENCES SN_Orders(Order_ID)
12  );
```

Table created.

4. Payment data insertion

```
SQL> INSERT INTO Payment
 2  (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)
 3  VALUES
 4  (501, 101, 'UPI', SYSDATE, 'Successful', 1898.00);
```

1 row created.

```
SQL>
SQL> INSERT INTO Payment
 2  (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)
 3  VALUES
 4  (502, 102, 'Credit Card', SYSDATE, 'Successful', 1299.00);
```

1 row created.

```
SQL>
SQL> INSERT INTO Payment
 2  (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)
 3  VALUES
 4  (503, 103, 'Debit Card', SYSDATE, 'Failed', 1499.00);
```

1 row created.

```
SQL>
SQL> INSERT INTO Payment
 2  (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)
 3  VALUES
 4  (504, 104, 'UPI', SYSDATE, 'Successful', 2499.00);
```

1 row created.

5. Manage payment transactions

5.1 Display successful payments

```
SQL> SELECT Payment_ID, Order_ID, Payment_Mode, Payment_Amount, Payment_Status
 2  FROM Payment
 3  WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
501	101	UPI	1898	Successful
502	102	Credit Card	1299	Successful
504	104	UPI	2499	Successful

5.2 Display pending payments

```
SQL> SELECT Payment_ID, Order_ID, Payment_Mode, Payment_Amount, Payment_Status
  2  FROM Payment
  3  WHERE Payment_Status = 'Pending';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
505	105	Cash on Delivery	999	Pending

6. Modify payment status

```
SQL> UPDATE Payment
  2  SET Payment_Status = 'Successful'
  3  WHERE Payment_ID = 503;
```

1 row updated.

```
SQL>
SQL> UPDATE Payment
  2  SET Payment_Status = 'Failed'
  3  WHERE Payment_ID = 505;
```

1 row updated.

7. Analyze payment methods

```
SQL> SELECT
  2  Payment_Mode,
  3  COUNT(*) AS Number_Of_Payments
  4  FROM Payment
  5  GROUP BY Payment_Mode;
```

PAYMENT_MODE	NUMBER_OF_PAYMENTS
UPI	2
Credit Card	1
Debit Card	1
Cash on Delivery	1

```
SQL> SELECT
  2  Payment_Mode,
  3  SUM(Payment_Amount) AS Total_Amount
  4  FROM Payment
  5  GROUP BY Payment_Mode;
```

PAYMENT_MODE	TOTAL_AMOUNT
UPI	4397
Credit Card	1299
Debit Card	1499
Cash on Delivery	999

8. Payment transaction report

```
SQL> SELECT
  2     P.Payment_ID,
  3     P.Order_ID,
  4     P.Payment_Mode,
  5     P.Payment_Date,
  6     P.Payment_Amount,
  7     P.Payment_Status,
  8     OD.Product_ID,
  9     OD.Quantity,
 10     OD.Unit_Price,
 11     OD.Subtotal
 12 FROM Payment P
 13 LEFT JOIN ORDER_DETAILS OD
 14 ON P.Order_ID = OD.Order_ID
 15 ORDER BY P.Payment_ID;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
PAYMENT_STATUS	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
501 Successful	101	UPI	29-SEP-26	1898

502 Successful	102	Credit Card	29-SEP-26	1299
503 Successful	103	Debit Card	29-SEP-26	1499

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
PAYMENT_STATUS	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
504 Successful	104	UPI	29-SEP-26	2499

505 Failed	105	Cash on Delivery	29-SEP-26	999
---------------	-----	------------------	-----------	-----